

MACHINE LEARNING ENGINEERING

FASE 5 | AULA 04 -

PADRONIZAÇÃO DE AMBIENTES COM PYPROJECT.TOML

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	10
REFERÊNCIAS.....	11

EMSE

O QUE VEM POR AÍ?

Depois de nos dedicar a entender os desafios de criar ambientes replicáveis, podemos pensar não apenas na estrutura das bibliotecas e sim em preparar nosso projeto para execução de forma completa, desde a instalação até a operação automatizada.

E depois, ainda precisamos pensar no futuro do nosso projeto: como vamos criar versões atualizadas do trabalho que estamos fazendo? Como vamos disponibilizar estas versões para uso?

HANDS ON

Já vimos em aulas anteriores o arquivo `pyproject.toml`, que é uma espécie de “coração operacional” dos projetos em Python. Agora, vamos nos aprofundar nele, para entender como preparar um ambiente completo que vá além do simples gerenciamento de dependências.

Vamos ver também como usar algumas características do git para nos ajudar com o controle de versões do projeto e algumas boas práticas de nomenclatura de versões.

SAIBA MAIS

O pyproject.toml em mais detalhes

Nas primeiras aulas, quando discutimos os problemas de reprodutibilidade em ambientes Python, apresentamos o arquivo `pyproject.toml` como o modo moderno de catalogar as dependências. Mas esse arquivo serve para muito mais coisas do que apenas isso, ele foi pensado como uma solução completa para operação de aplicações. Vamos abrir o arquivo e entender, seção por seção, o que ele contém, o que cada parte significa e como usá-lo em projetos reais de ciência de dados.

E vamos também usar a ferramenta `uv` para fazer as tarefas que são automatizadas, como a instalação de repositórios e outras que vamos ver posteriormente.

O `pyproject.toml` usa o formato TOML (Tom's Obvious, Minimal Language), uma linguagem de configuração projetada para ser legível por humanos e com menos complexidade que outros formatos como json e yaml. A sintaxe é simples: chaves e valores, organizados em seções delimitadas por colchetes (`[secao]`) e subseções (`[secao.subsecao]`).

Um `pyproject.toml` típico é dividido em três grandes blocos:

```
[build-system] # Como o projeto é construído
[project]      # Metadados e dependências do projeto
[tool.nome]    # Configurações de ferramentas específicas
```

Na seção `[build-system]`, definimos qual o mecanismo de build do projeto, servindo para aplicações e software que serão distribuídos para ser componentes de outros projetos ou para projetos mais complexos de aplicações como servidores web, APIs ou bibliotecas. Para projetos de ciência de dados que não serão publicados no PyPI (o repositório central na internet, onde usualmente ficam todas as bibliotecas que instalamos e importamos), que são a grande maioria, a escolha desse mecanismo não importa na prática. Por este motivo, podemos manter o valor padrão que é definido ao usar `uv init`, que já configura essa seção automaticamente.

Na seção [project], começamos a definir de forma mais detalhada o nosso projeto. Isso porque é nessa seção que colocamos os metadados do nosso projeto, e também onde definimos as dependências do projeto.

Por exemplo, podemos definir aqui algumas informações sobre o projeto:

```
[project]
name = "analise-churn"
version = "0.1.0"
description = "Modelo de predição de churn para clientes"
readme = "README.md"
requires-python = ">=3.11"
```

Código-fonte 1 – Informações sobre o projeto (Python)

Fonte: Elaborado pelo autor (2026)

É nessa seção que definimos qual a versão do Python que pode ser usada para o projeto. Olhando na descrição anterior, em requires-python, definimos que pode ser qualquer versão depois da 3.11. Para definir que só podemos usar versões entre 3.11 e 3.12, podemos colocar esta linha como requires-python = ">=3.11,<3.13"

Também é dentro de [project] que encontramos a lista de dependências do projeto. Mas é importante lembrar que o gerenciamento de dependências deve ser feito, preferencialmente, pelo uv add <pacote>, porque o uv já consegue registrar corretamente as versões de biblioteca e validar possíveis conflitos.

Vamos ver um exemplo de como as dependências são declaradas:

```
dependencies = [
    "pandas>=2.0,<3.0",
    "scikit-learn>=1.3",
    "matplotlib>=3.7",
    "xgboost>=2.0",
    "joblib>=1.3",
]
```

Código-fonte 2 – Como as dependências são declaradas (Python)

Fonte: Elaborado pelo autor (2026)

Neste caso, vemos o nome da dependência e a versão de cada, além de um símbolo indicando a restrição de versão, usando os operadores:

OPERADOR	SIGNIFICADO	EXEMPLO
<code>>=</code>	Maior ou igual	<code>pandas>=2.0</code>
<code><</code>	Menor que	<code>pandas<3.0</code>
<code>==</code>	Exatamente	<code>pandas==2.1.4</code>
<code>~=</code>	Compatível com	<code>pandas~=2.1</code>
<code>,</code>	Combinação (E)	<code>pandas>=2.0,<3.0</code>

Tabela 1 – Nome da dependência, versão e símbolo
Fonte: Elaborado pelo autor (2026)

Mas, vamos reiterar: não devemos mexer nessa parte do arquivo diretamente, sob pena de quebrar a estrutura de dependências de nossa aplicação. Mas é importante saber como ler as dependências e suas versões.

E na seção `[tool]` temos uma funcionalidade muito importante para operacionalizar o processo de produtização do nosso modelo, que é a possibilidade de criar comandos personalizados para executar os passos do pipeline, com treino, testes e inicialização da aplicação.

Vamos ver como fazer isso de duas formas. A primeira é usando a seção `[tool]`, como falamos anteriormente.

```
[tool.uv]
[tool.uv.scripts]
treinar = "python scripts/treinar.py"
notebook = "jupyter lab"
lint = "ruff check src/"
testes = "pytest tests/ -v"
```

Código-fonte 3 – Seção `[tool]` (Python)
Fonte: Elaborado pelo autor (2026)

Se criarmos os comandos dessa forma, não precisamos decorar como rodar o script correto; podemos apenas rodar o comando `uv run treinar` para treinar o modelo e `uv run testes` para executar os testes da aplicação.

Isso é extremamente poderoso para a automatização dos processos e para ajudar na documentação; padronizamos como cada comando tem que ser rodado, sem chances de erros de digitação e de modo facilitado para a operação do CI/CD.

A segunda forma é usar a sintaxe padrão do `pyproject.toml`, mas ela exige que o projeto siga uma regra: todo o código-fonte deve estar dentro do path `src/`. Se seu projeto está dessa forma, você pode usar a seção `[project.scripts]`, como a seguir:

```
[project.scripts]
treinar = "src.modelo:treinar"
avaliar = "src.modelo:avaliar"
preprocessar = "src.pipeline:executar_preprocessamento"
```

Código-fonte 4 – Seção `[project.scripts]` (Python)

Fonte: Elaborado pelo autor (2026)

Neste caso, o script executa diretamente uma função dentro de um módulo. No caso anterior de treinar ao executar o `uv run treinar`, o `uv` vai executar a função `treinar` dentro de `src/modelo.py`.

Versionamento do projeto

Como padrão em projetos de software, qualquer que seja sua natureza, o `git` é a ferramenta comumente usada para fazer o controle do código. O uso do `git` está disseminado e, atualmente, saber como usar o `git` é um pré-requisito para trabalhar com construção de software. O conhecimento de como fazer commits, mudar de branch, fazer pull e push e resolver conflitos de arquivos deve ser parte básica do conhecimento de qualquer pessoa que trabalha com software, e não é exceção para a área de dados.

Mas, o que devemos versionar? Por padrão, versionamos apenas o código e ignoramos os artefatos, como os modelos treinados e as origens de dados de grande volume (como arquivos e dumps de banco de dados). Este tipo de artefato deve ser armazenado em algum tipo de espaço que esteja preparado para este tipo

de arquivo, como drives de rede ou buckets de algum serviço de cloud. Vamos para alguns exemplos:

ARQUIVO	VERSIONAR?	MOTIVO
pyproject.toml	Sim	É a declaração de configurações do projeto
uv.lock (arquivos .lock)	Sim	Garante reprodutibilidade exata
.python-version	Sim	Fixa a versão do interpretador
.venv/	Não	Gerado localmente, contém binários
requirements.txt gerado	Depende	Apenas se necessário para compatibilidade com ferramentas legadas

Tabela 2 – O que devemos versionar?

Fonte: Elaborado pelo autor (2026)

E, para fazer o gerenciamento de versões de um projeto, podemos usar as git tags. Usualmente, sempre que um projeto está em uma versão fechada, marcamos o commit da entrega com um tag de versão, por exemplo, git tag 1.0 para marcar o commit atual com a versão 1.0. Depois, para publicar esta tag em um repositório remoto, podemos usar o comando git push --tags .

O QUE VOCÊ VIU NESTA AULA?

O arquivo `pyproject.toml` é mais importante do que parece à primeira vista: além de ser o controle de dependências, ele também configura o comportamento da operação do nosso processo de dados. Ao permitir a criação de comandos específicos, reduzimos muito a chance de erros no processo e garantimos que os comandos sempre sejam executados de forma certa. Com isso, temos uma ferramenta que nos ajuda a controlar as dependências, reproduzir ambientes e equacionar as configurações e características dos ambientes.

Além disso, vimos algumas ideias para melhorar a qualidade do versionamento dos nossos projetos.

REFERÊNCIAS

MAHAMULKAR, P. **Machine Learning Operations (MLOps) For Beginners**. 2024. Disponível em: <https://towardsdatascience.com/machine-learning-operations-mlops-for-beginners-a5686bfe02b2/>. Acesso em: 02 abril 2026.

MLOPS. **Machine Learning Operations**. 2026. Disponível em: <https://ml-ops.org/>. Acesso em: 21 abr. 2026. 02 abril 2026.

POTE, S. **Machine Learning in Production**. [s.l.]: BPB Publications, 2023.

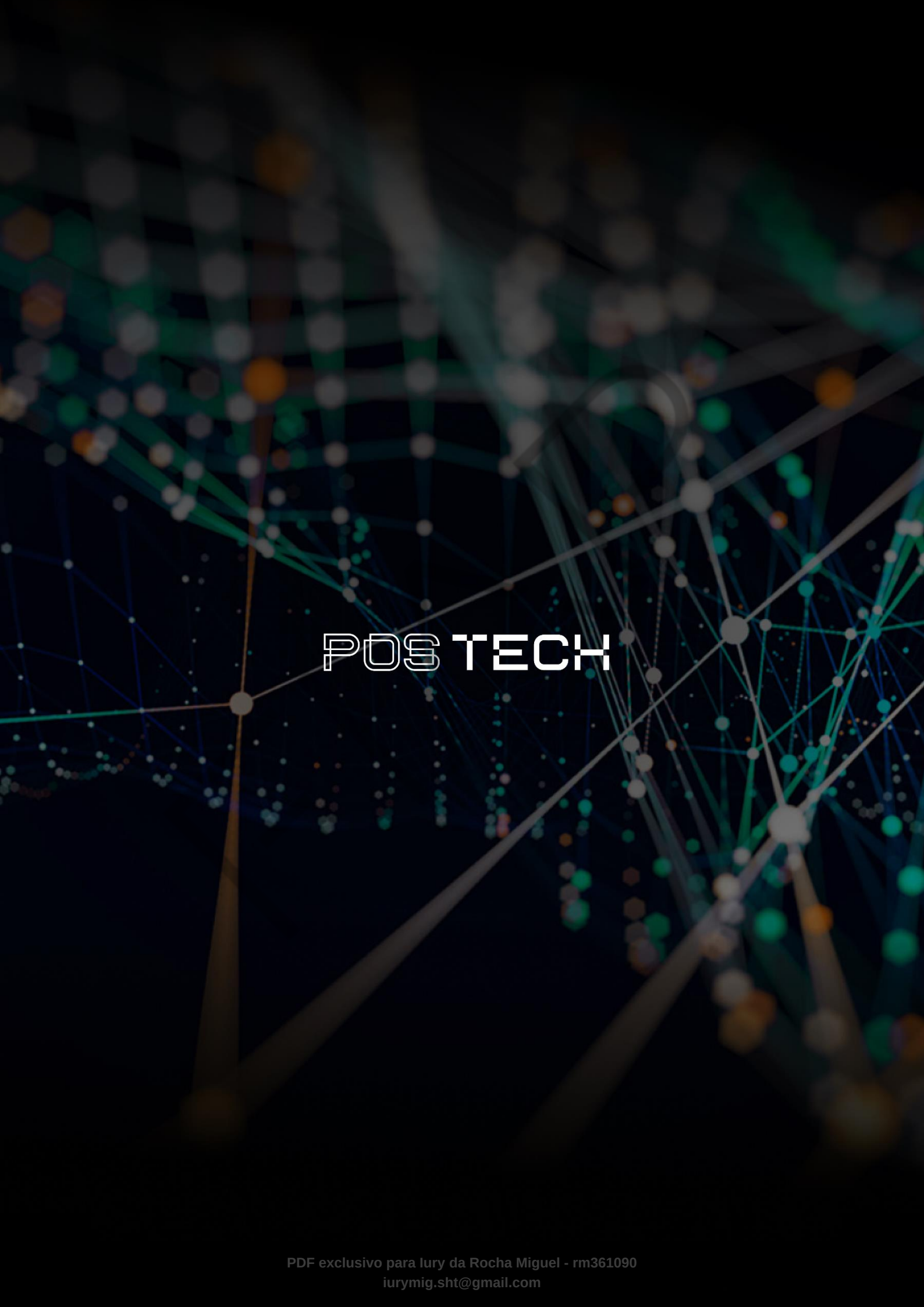
PYTHON PACKAGING USER GUIDE. **Escrevendo seu pyproject.toml**. 2026. Disponível em: <https://packaging.python.org/pt-br/latest/guides/writing-pyproject-toml/>. Acesso em: 22 abr. 2026.

UV. **uv**. 2026. Disponível em: <https://docs.astral.sh/uv/>. Acesso em: 22 abr. 2026.

PALAVRAS-CHAVE

pyproject.toml. Configuração central. Dependências. Versionamento.

EMENDAS



POSTECH